GUARDIAN MISSION

Integrated Project Report

Battery management, sensor-based mission supervision, vehicle kinematics and 3D visualization with working vehicle lights in Simulink

Prepared by

NAGY MOHAMED ELBAZ

October 2026

1. Introduction and objectives

The Guardian Mission project joins three separately developed Simulink models into one closed-loop system: a battery digital twin, a sensor-processing and decision unit, and a vehicle that drives and is shown in a 3D scene. Distance sensors tell the vehicle what to do, the vehicle's motion draws current from the battery, and the battery's condition in turn decides how the vehicle is allowed to move.

The objectives of the project were to:

- read three distance sensors and turn them into a driving state (forward, turn left, turn right, stop);
- estimate the battery state of charge (SOC), state of health (SOH) and terminal voltage from the current drawn;
- supervise the whole mission with a state machine that reduces speed on low power and stops the vehicle safely when the battery is critical or the emergency stop is pressed;
- convert the speed and turn commands into a vehicle position and heading (kinematics); and
- show the result in a 3D scene with a realistic car whose brake lights, headlights and turn signals respond to what the car is doing.

2. System overview

Figure 1 shows the top level of Integrated_Project. Data flows from left to right in four stages: scenario inputs, sensor processing, mission supervision and battery, and finally kinematics with the 3D model.

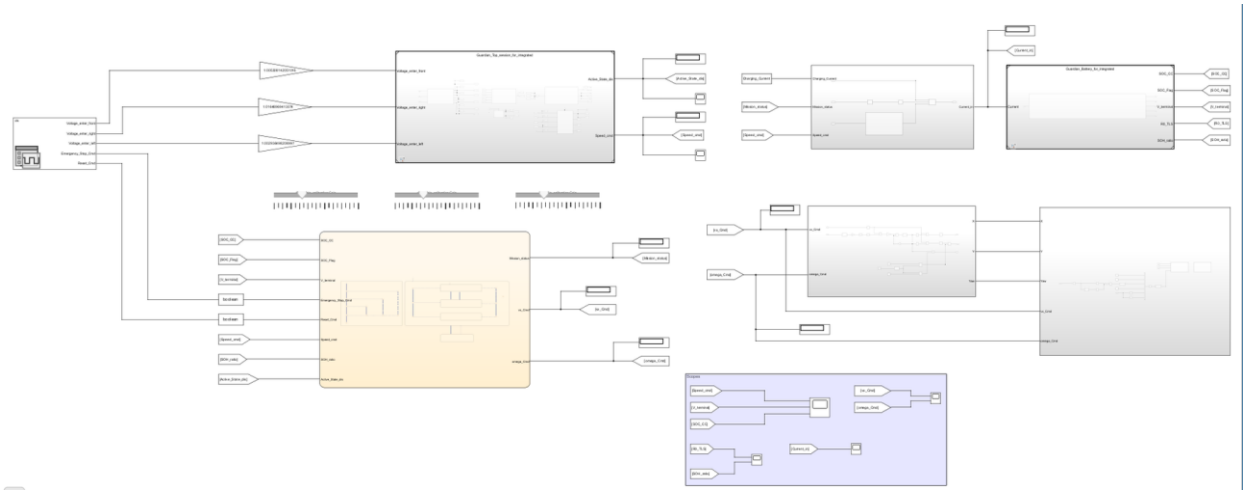


Figure 1. Top level of the Integrated_Project model.

Block	Role in the system
Signal Editor (ds)	Supplies the test scenario: three sensor voltages (front, right, left), Emergency_Stop_Cmd and Reset_Cmd.
Calibration gains	Three gains (front 1.0053, right 1.0165, left 1.0029) correct the sensor voltages before processing. Sliders allow tuning.

Block	Role in the system
Rover Model (Guardian_Top_session_for_integrated)	Referenced model that turns the sensor voltages into distances, an active state (Active_State_dis) and a speed command (Speed_cmd).
Mission_Supervisor	Stateflow chart. Combines the active state with battery information and the buttons to produce Mission_status, vx_Cmd and omega_Cmd.
Generate Current	Converts Speed_cmd and Mission_status into the load current drawn from the battery, or the charging current while the vehicle is stopped for return-to-base.
Battery Model (Guardian_Battery_for_integrated)	Referenced battery digital twin. Outputs SOC_CC, SOC_Flag, V_terminal, R0_TLS and SOH_ratio.
Kinematics	Turns vx_Cmd and omega_Cmd into the vehicle position X, Y and heading Yaw.
3D_Model	Draws the car in the 3D scene and drives its lights.
Scopes	Display speed, voltage, SOC, SOH, current, vx_Cmd and omega_Cmd for analysis.

3. What was needed to build the project

3.1 Software

- MATLAB R2025b and Simulink, with Stateflow for the Mission_Supervisor and the debounce logic.
- The Simulink 3D simulation blocks (Simulation 3D Scene Configuration and Simulation 3D Vehicle with Ground Following), which run the Unreal Engine scene. These come with MathWorks' automotive and driving toolboxes, so the exact license should be confirmed with the ver command.
- A computer able to run the 3D window smoothly (a graphics card helps).

3.2 Input files

File	Purpose
Guardian_Battery_DriveCycle_Current.csv	Battery drive-cycle current (time and current) for the original battery test.
Guardian_Battery_OCV_SOC_Table.csv	Open-circuit voltage against SOC lookup table.
Guardian_Battery_for_integrated.slx	Battery digital twin (referenced model).
Guardian_Top_session_for_integrated.slx	Sensor, debounce and state-decision model (referenced model).
SensorProcessing_session_for_integrated.slx	Single-sensor conversion from voltage to distance and close flag (used three times).
Integrated_Project.slx	The integrated top-level model.

File	Purpose
Integrated_Project_Script.m	Prepares all workspace data and builds the scenario file.

3.3 Setup script

One MATLAB script prepares everything the model needs before it runs. It deliberately contains no sim() call or set_param() loop, so the model runs purely from Simulink. It:

- defines the sample time Ts = 0.1 s and the first-order current response (time constant 2 s);
- defines the mission thresholds and speeds (Table 1);
- reads the two CSV files and defines the battery equivalent-circuit constants;
- creates the bus objects used by the battery and the sensor model; and
- builds the test scenario (sensor voltages, emergency stop and reset pulses) and saves it as scenario_FINAL_ceremony.mat.

Parameter	Value	Meaning
Ts	0.1 s	Fixed sample time of the whole model
SOC_Warn	30 %	Below this the mission goes to LOW_POWER
SOC_Critical	5 %	Below this the vehicle is stopped (RTB_STOP)
SOH_Min	90 %	Below this the mission goes to LOW_POWER
V_min	2.5 V	Below this the vehicle is stopped (RTB_STOP)
vx_FORWARD	0.5 m/s	Cruise speed in NOMINAL
vx_REDUCED	0.2 m/s	Speed in LOW_POWER and CELL_FAULT
omega_TURN	0.5 rad/s	Magnitude of the turn command
Charging_Current	-25 A	Charging current while stopped for return-to-base
Q_nominal	50 Ah	Battery capacity (LiFePO4 cell, 3.2 V). Reduced to 12 Ah only for a fast demonstration.

Table 1. Main parameters defined by the setup script.

4. Main subsystems

4.1 Sensor processing and decision (Rover Model)

Each of the three sensors (front, right, left) is handled by the same small model. A voltage between 0 V and 5 V is limited by a saturation block and converted to a distance in centimetres; a switch then sets a “close” flag. The scenario converts distance to voltage with the table 0 to 5 V against 400, 300, 200, 100, 50 and 10 cm, so a higher voltage means a closer obstacle.

A Stateflow debounce-and-heal block protects the decision from noise. The button logic needs N_confirm = 3 consecutive samples before a button change is accepted, and the sensor logic treats a

reading outside 0 to 500 cm as invalid, holds the last good value and trusts the sensor again after N_heal = 3 valid samples.

The “brain” of this model picks one active driving state: FORWARD, IDLE, TURN_L, TURN_R or STOP. It outputs the name of the state (Active_State_dis) and a speed command (Speed_cmd). The values used are IDLE = 50, FORWARD = 100, TURN = 25 and STOP = 0.

4.2 Mission supervisor

The Mission_Supervisor chart decides how the vehicle may move. Inside the “operating” super-state it has three modes, and a fourth state stops the vehicle.

State	Mission_status	Behaviour
NOMINAL	1	Normal driving at vx_FORWARD (0.5 m/s).
LOW_POWER	2	Entered when SOC_CC falls below SOC_Warn or SOH_ratio below SOH_Min. Driving continues at the reduced speed vx_REDUCED (0.2 m/s).
CELL_FAULT	3	Entered when the battery estimator raises SOC_Flag. Driving continues at reduced speed.
RTB_STOP	4	Entered from any operating mode when Emergency_Stop_Cmd is true, SOC_CC < SOC_Critical or V_terminal < V_min. The vehicle stops (vx_Cmd = 0, omega_Cmd = 0) and charging starts automatically. Reset_Cmd returns to the previous mode (history state); if the cause is still present the supervisor goes straight back to RTB_STOP, which is why an early reset “bounces back”.

Table 2. States of the Mission_Supervisor.

A small function, StateToKinematics, turns the active driving state into the two kinematic commands: in FORWARD, vx = vx_scale × Speed_cmd / 100 and omega = 0; in a left turn, vx = 0 and omega = +omega_TURN; in a right turn, vx = 0 and omega = −omega_TURN; in STOP, both are zero. So a positive omega_Cmd always means a left turn and a negative one a right turn.

4.3 Current generation and battery

The Generate Current subsystem maps the speed command to a load current through a lookup table: IDLE 20 A, FORWARD 40 A, TURN 10 A and STOP 0 A. While Mission_status is RTB_STOP it selects the charging current (−25 A) instead. The result passes through a first-order filter with a time constant of 2 s, so the current changes smoothly, and then goes to the battery.

The battery model is a digital twin of an EVE LF50K, 3.2 V, 50 Ah LiFePO4 cell. Its internal parts are:

- **Coulomb counting** gives SOC_CC by integrating the current, and provides the SOC_Flag used by the supervisor;
- **Open-circuit voltage with hysteresis** uses the OCV–SOC table with a noise filter (FR-3);
- **Two RC branches** (R1 = 1.6 mΩ, C1 = 17 kF and R2 = 0.35 mΩ, C2 = 5.2 kF) with R0 = 0.7 mΩ model the voltage dynamics and give V_terminal;
- **Extended Kalman Filter** estimates SOC from current and voltage; and
- **Total least squares** estimates the internal resistance R0_TLS and the health ratio SOH_ratio.

5. Kinematics: from inputs to outputs

The Kinematics subsystem has two inputs and three outputs.

Port	Direction	Description
vx_Cmd	Input 1	Forward speed command in m/s from the Mission_Supervisor.
omega_Cmd	Input 2	Turn command (+0.5 left, −0.5 right, 0 straight) from the Mission_Supervisor.
X	Output 1	Position along the road in metres.
Y	Output 2	Sideways position in metres.
Yaw	Output 3	Heading of the car in radians.

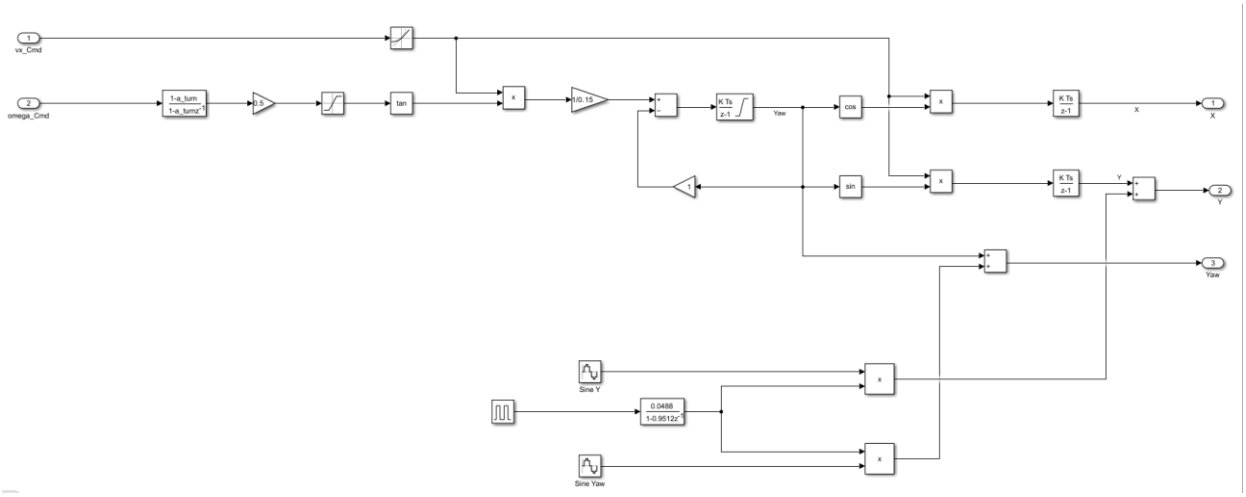


Figure 2. The Kinematics subsystem.

5.1 Step-by-step signal flow

1. **Smooth the speed.** vx_Cmd goes through a Rate Limiter (rising 0.5 m/s², falling −1 m/s²). The car accelerates gently and brakes a little faster instead of jumping to the commanded speed. The output is the real speed v.
2. **Smooth the steering command.** omega_Cmd goes through a discrete filter $(1 - a)/(1 - a \cdot z^{-1})$ with a = exp(−Ts/0.5) ≈ 0.819, a time constant of 0.5 s, so the steering eases in and out.
3. **Form the steering angle.** A gain of 0.5 and a saturation of ±0.5 rad give the steering angle δ.
4. **Apply the bicycle model.** The yaw-rate command is v · tan(δ) / L with wheelbase L = 0.15 m (the 1/0.15 gain). A car can only turn while it moves, so the turn rate depends on the speed.
5. **Close a heading-return loop.** The Sum block subtracts the current Yaw (feedback gain 1) from the yaw-rate command, so the heading returns to straight when the steering returns to zero.
6. **Integrate the heading.** A Discrete-Time Integrator with output limits of ±0.4 rad gives Yaw. The limit means the car can never spin round.
7. **Integrate the position.** Yaw goes through cos and sin and is multiplied by v. Two integrators (Ts = 0.1 s) give X = ∫ v cos(Yaw) dt and Y = ∫ v sin(Yaw) dt.
8. **Add the weaving manoeuvre.** A Pulse Generator (period 1000 s, width 24 s, delay 30 s) is smoothed by a filter with a 2 s time constant and multiplies two sine waves (amplitude 0.6 and

0.5, frequency 0.6 rad/s, the second shifted by 90°). The results are added to the Y and Yaw outputs, after the feedback loop, so the car swings right and left while moving forward between 30 s and 54 s and then straightens out. No MATLAB Function block is used.

Equations: Yaw′ = v·tan(δ)/L − Yaw, X′ = v·cos(Yaw), Y′ = v·sin(Yaw), with a limit of ±0.4 rad on Yaw.

5.2 Why the car spun and how it was fixed

The first version integrated the turn command directly into the heading. A turn lasting 60 s at 0.5 rad/s adds 30 rad, almost five complete rotations, so the car spun and then drove off sideways. The fix has four parts: a heading-return feedback, a limit on the heading, a bicycle model so the car cannot turn while stopped, and shorter turns in the scenario.

6. 3D model and vehicle lights

6.1 The 3D model

The 3D_Model subsystem contains the Simulation 3D Scene Configuration block (a straight road scene) and the Simulation 3D Vehicle with Ground Following block, set to a muscle-car body to resemble a Dodge Challenger. X, Y and Yaw from the Kinematics go straight into the vehicle block. The block adjusts height, roll and pitch to the road and turns each wheel from the distance the car moves, so the wheels spin and steer automatically.

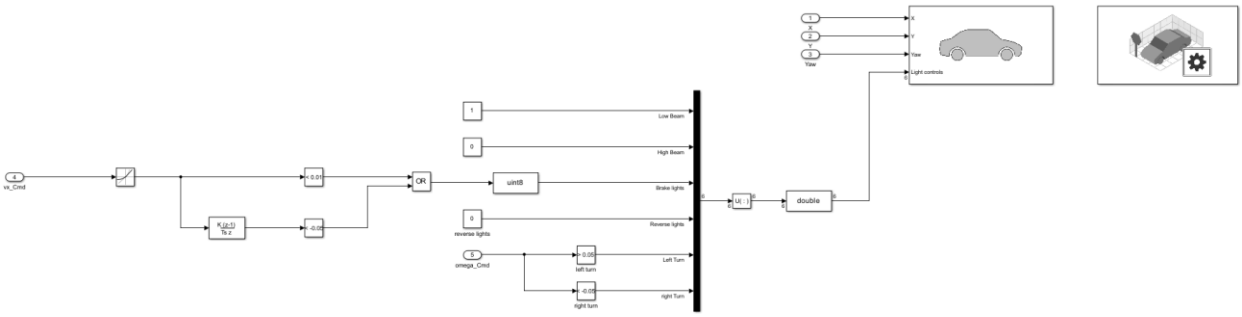


Figure 3. The 3D_Model subsystem, including the light-control logic.

6.2 How the lights are controlled

Enabling light controls on the vehicle block adds an input called Light controls. It does not take separate signals. It takes one row vector of six elements ([1×6]) in which each element switches one lamp on (non-zero) or off (zero). The order of the six positions was found by testing: a constant vector with a single 1 was applied to the port and the lamp that lit was noted.

Position	Lamp	Signal driving it
1	Low beam	Constant 1 (always on).
2	High beam	Constant 0 (off).
3	Brake lights	ON when the car is stopped (v < 0.01 m/s) OR when it is slowing down (acceleration < −0.05 m/s²).

Position	Lamp	Signal driving it
4	Reverse lights	Constant 0 (the car never reverses).
5	Left turn signal	omega_Cmd > 0.05 (left turn).
6	Right turn signal	omega_Cmd < −0.05 (right turn).

Table 3. Meaning of the six elements of the Light controls vector.

6.3 Step-by-step build

1. Enable light controls in the Light Controls tab of the vehicle block. The Light controls input port appears.
2. Brake logic: pass vx_Cmd through a Rate Limiter with the same settings as in the Kinematics (0.5 and −1), so the lights follow the real speed rather than the instant command.
3. Branch the smoothed speed. One branch goes to a Compare To Constant (< 0.01) to detect “stopped”. The other goes to a Discrete Derivative (sample time 0.1 s) and a Compare To Constant (< −0.05) to detect “slowing down”.
4. Combine the two results with a Logical Operator (OR), then convert the result to uint8 with a Data Type Conversion block.
5. Turn signals: send omega_Cmd to two Compare To Constant blocks, one for > 0.05 (left) and one for < −0.05 (right), both with output type uint8.
6. Add Constant blocks for low beam (1), high beam (0) and reverse (0), all of type uint8.
7. Join the six signals in the order of Table 3 with a Mux block.
8. A Mux gives a plain vector, but the port needs a [1×6] row, so add a Reshape block with output dimensionality Customize and dimensions [1, 6].
9. Add a Data Type Conversion (double) and connect it to the Light controls port. The line should be labelled [1x6] with Signal Dimensions displayed.

6.4 Light appearance parameters

The values below only control how the lamps look. Switching is done by the signals above. The turn signals blink by themselves, because the block’s own period and pulse width are used, so no extra pulse generator is needed.

Parameter	Value used
Headlight colour [R, G, B]	[1, 1, 0.9] (slightly warm white)
High beam / low beam intensity	5000 cd / 1000 cd (can be raised for a darker scene)
Beam cone half angle	35° for both beams
Beam orientation [pitch, yaw]	[−5, 0] degrees for both headlights
Brake and reverse intensity	400 cd/m²
Turn signal intensity	5000 cd/m² (raise it for night scenes)
Turn signal period and pulse width	1 s and 50 %



Figure 4. The car in the 3D scene with its brake lights on.

7. Results

Figure 5 shows the speed command and the turn command over a 1400 s run. The speed command stays at 0.5 m/s and falls to zero only at three moments: during the right turn at about 20 s, during the left turn at about 60 s, and during the emergency stop between about 250 s and 260 s. omega_Cmd shows the two turns as short pulses: −0.5 rad/s at about 20 s (right) and +0.5 rad/s at about 60 s (left).

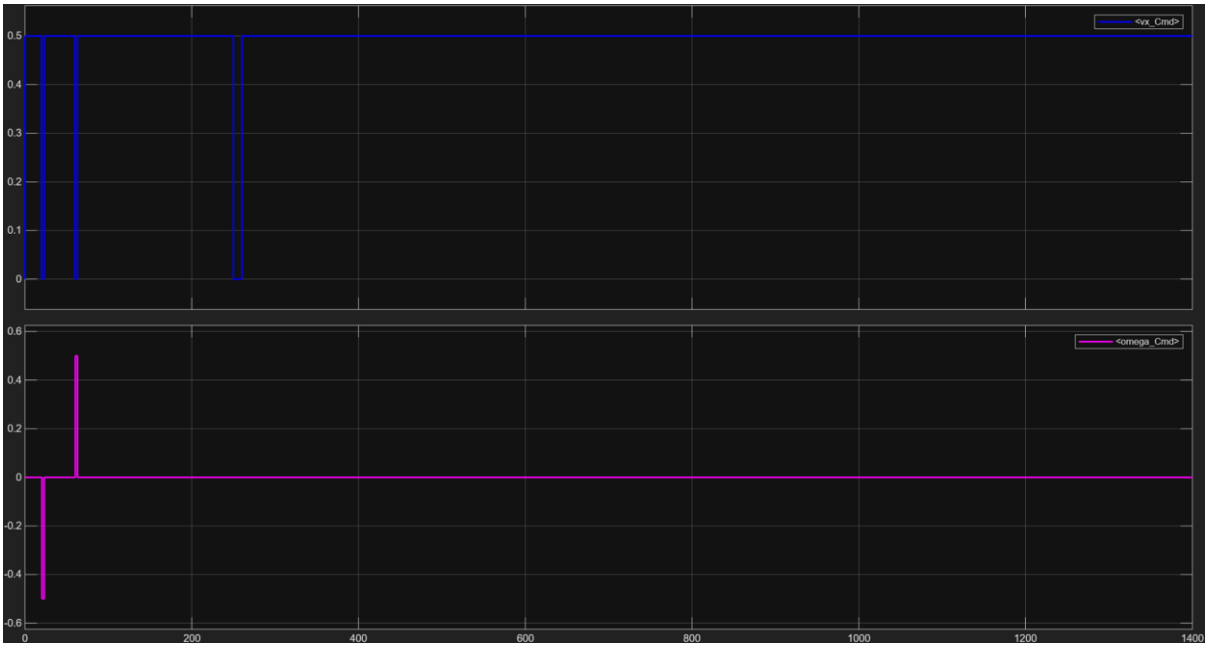


Figure 5. vx_Cmd (top) and omega_Cmd (bottom).

Figure 6 shows the current drawn from the battery. It settles at 40 A in forward driving and dips towards the 10 A turn value at the two turns, but only reaches about 17 A because the current responds with a 2 s time constant and the turns are short. During the emergency stop the current becomes −25 A, which is the automatic charging, and it returns to 40 A when the reset is accepted at 260 s.

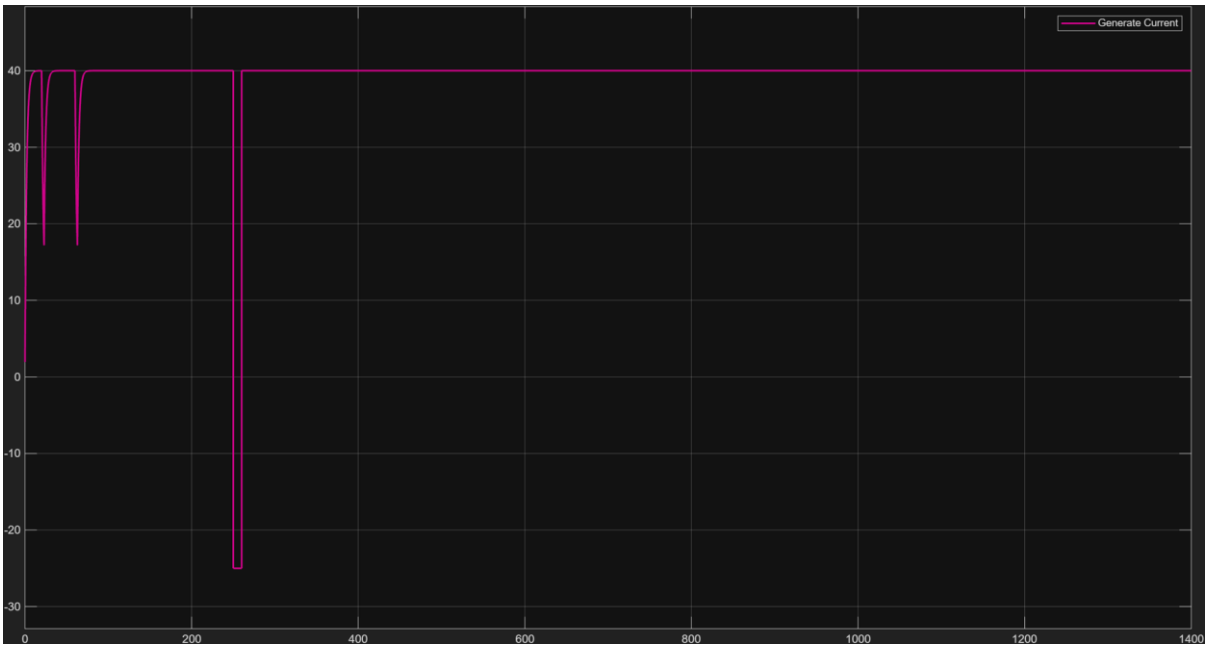


Figure 6. Generated battery current.

Figure 7 shows the speed command from the sensor side and the battery response over the same 1400 s. Speed_cmd stays at 100 (FORWARD) and drops to 25 (TURN) at the two turns at about 20 s and 60 s. It does not change at the emergency stop, because the stop is imposed by the Mission_Supervisor and not by the sensors.

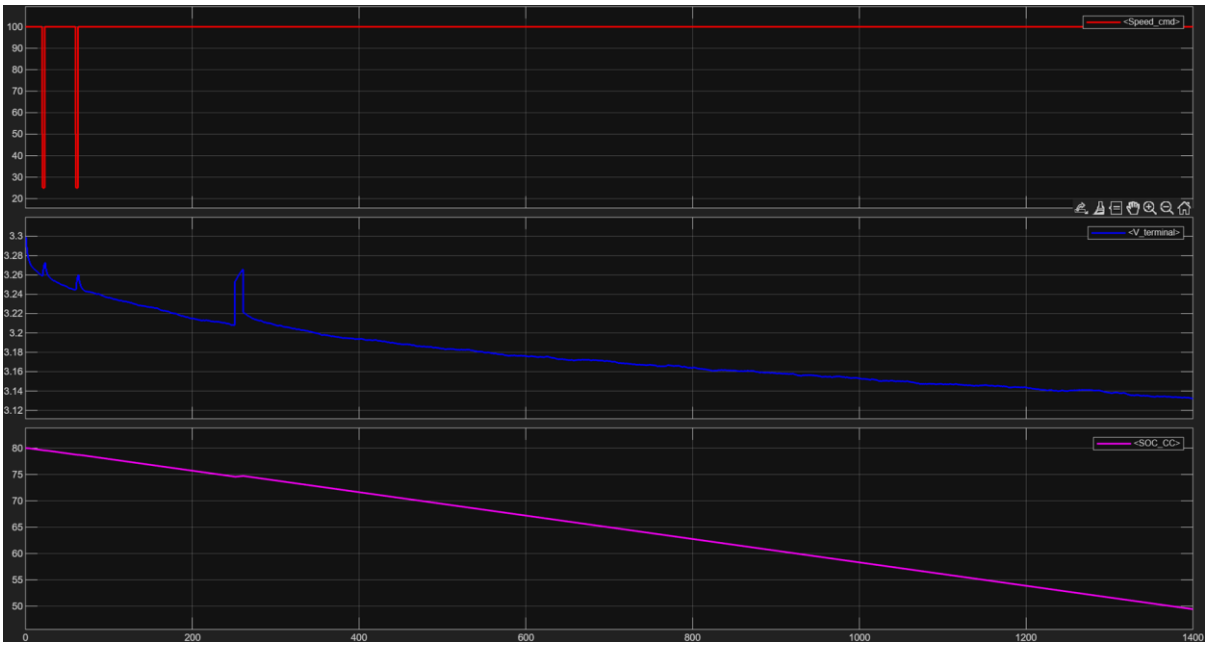


Figure 7. Speed_cmd, battery terminal voltage and SOC_CC.

The terminal voltage starts near 3.30 V and falls slowly to about 3.13 V as the cell discharges. Small upward steps appear at the two turns, where the load current is lower, and a larger jump to about 3.26 V appears during the emergency stop, when the charging current of −25 A flows. SOC_CC falls almost in a straight line from 80 % to about 49.5 % over the run, with a small flat section around 250 to 260 s from the charging. This agrees with the hand calculation: 40 A for 1400 s is about 15.6 Ah, which is about 31 % of the 50 Ah capacity.

In this run the battery stays above its thresholds for the whole 1400 s (SOC_CC is still about 49.5 %, well above SOC_Warn = 30 %), so the supervisor stays in NOMINAL except for the emergency stop. To see LOW_POWER, CELL_FAULT and RTB_STOP within about 900 s, the capacity is temporarily lowered to 12 Ah for the demonstration scenario. The expected milestones are LOW_POWER at about 630 s, CELL_FAULT at about 846 s and RTB_STOP at about 900 s, followed by a premature reset at 901 s that must bounce back and a successful reset at 950 s.

8. Problems met and how they were solved

Problem	Cause	Solution
Car spun, then drove to the left	The turn command was integrated into the heading for 60 s, giving several full rotations.	Heading-return loop, ±0.4 rad limit, bicycle model and short turns.
Fixed-step solver error with 1/8000 s	Live engine sound needs a very small step, which conflicts with the 0.1 s battery and sensor models.	Step kept at 0.1 s; sound is generated afterwards from the logged speed.
“Discrete solver cannot be used”	The model contains continuous states.	Fixed-step ode1 at 0.1 s in the main and the referenced models.
Terrain error in the 3D vehicle	The car left the area where the scene could place it on the ground.	Straight-road scene, start at [0, 0, 0], and keep the motion inside the scene.
Light controls error “size must be [1 x 6]”	A Mux gives a plain vector, not a row.	A Reshape block set to [1, 6].
Data type mismatch at the Mux	The OR block outputs boolean while the other inputs were uint8.	A Data Type Conversion to uint8 after the OR.
Sample-time warning between two models	A 0.1 s subsystem feeds a 1 s model.	A Rate Transition block (or the same sample time).
Wrong lamp order	The order of the six elements was assumed.	Tested each position with a constant vector.

9. Limitations and recommendations

- **Turns are almost on the spot.** The supervisor sets vx = 0 during a turn. Because the bicycle model needs forward speed, the car hardly changes heading during the 3 s turn pulses. A small forward speed during turns (for example 0.2 m/s) would give a visible arc and let the turn signals match a real manoeuvre.
- **Turns are short.** The scenario turns last 3 s, so the turn signals blink only about three times. Making the turns 15 s long makes them clearly visible.
- **Live sound.** Playing engine sound inside Simulink needs a much smaller step than the models allow. The recommended method is to log vx, create an engine sound file in MATLAB and play it with the screen recording.
- **Reference models.** Referenced models must share the same fixed step as the parent; changing one requires changing all.
- **Real-time display.** Simulation Pacing at 1× keeps the 3D scene in step with simulation time.

- **Possible extensions.** Hazard lights during the emergency stop, high beam in the dark, and obstacles in the 3D scene placed to match the sensor distances.

10. Conclusion

The project joins a battery digital twin, sensor-based decision logic and a mission supervisor into one closed loop and shows the outcome in a 3D scene. The kinematics turn the supervisor's speed and turn commands into smooth, stable motion that cannot spin, and a weaving manoeuvre adds visible right-and-left movement without any MATLAB Function block. The vehicle lights are driven by the real state of the car: brake lights when it slows or stops, turn signals from the sign of the turn command, and headlights always on. The result is a complete chain from sensor voltage to a moving, lit car in the 3D scene.